iPRA: smaller checkers and the independence of progress

Two measured improvements and a coherent nonstandard model of the remaining kernel

Prepared for Vladimir Reshetnikov

12 November 2026

Abstract

Two complete compact checking programs improve the current repository's empirical size envelope: one uses 251 significant Python tokens instead of 253; another uses 502 normalized bytes instead of 506. They are different programs, and neither improves the incumbent 275-node AST record. These are implementation refinements of the existing bi-interpretable theory, not a new smaller arithmetic axiom basis.

The principal mathematical result answers the repository's open progress-independence question. A staged tower of ultrapowers supports every finite scalar iterator body, with actual equality and coherent substitution, while $I(c, 0; u.u) = 1 \neq 0$ at a nonstandard count. All seven non-progress kernel rules are sound in the resulting model. Nevertheless the literal equation $E(y) := I(y, 0; u.u) = 0$ has a provable base and successor step, so its progress closure is not derivable without the eighth rule. The construction can additionally preserve the ordinary commutative-monoid laws for addition and $I(n, 0; u.Su) = n$. A separate finite-clock argument excludes every fixed finite successor cycle without induction.

The archive includes explicit rules, both ordinary interpretation composites, complete checker source, frozen comparisons, provenance, finite regression evidence, and reproducible LaTeX. No global minimality or new proof-assistant verification is claimed; the requested single construction below every existing size coordinate is not established.

Contents

1 Results and the exact comparison **2**

2 A complete proof presentation and its arithmetic meaning **2**

3 Independence of progress: a coherent model of scalar computation **5**

4 Finite scalar clocks without induction **9**

5 Smaller implementations of the complete calculus **11**

6 What the new mathematics permits us to conclude **13**

7 Further research questions **14**

A Transfer and the countable-model refinement **14**

B Complete compact checker listings **15**

1 Results and the exact comparison

The source snapshot is commit

`6ffc49f55d355b548f7a8e1530570aee5de086b3`.

It contains ninety research packages and three unfiled archives in `docs/incoming/`. The latter must be inspected: the top-level package table alone misses some of the best implementations. The prior source audit in `ipra_orbit_closure.zip` covers exactly the current Research tree, whose Git tree hash is

`76de155aec9da910a74320fd692ed9ed790515c9`.

The old audit commit and the present commit have this identical tree; the new intake archives were separately scanned [1, 6].

All measurements below use CPython 3.12.14, whole standalone modules, `ast.unparse` normalization and one final newline. Significant tokens exclude comments and layout; AST counts include every node of the reparsed normalized source. The measured checkers contain no docstrings. Imports, input handling, verification, and the exact success output `ACCEPT` remain inside the measurement.

Complete program	Tokens	AST nodes	Normalized bytes
Frozen incoming token control	253	280	599
New token profile	**251**	283	605
Frozen repository byte control	267	303	506
New parity companion	270	303	503
New byte profile	269	301	**502**
New strict public-input companion	421	471	869

The old coordinate envelope is $(253, 275, 506)$; the new envelope is $(251, 275, 502)$. These tuples combine different programs. In particular, the 275-node program remains an inherited implementation: no supplied new program has all three new scores. The byte improvement is $4/506$, about 0.79%, and the token improvement is $2/253$, also about 0.79%. These modest changes must not be described as a mathematical axiom deletion.

The observed envelope is an attained upper bound on the unknown minimum implementation size. It is not a theorem excluding smaller programs or different proof languages. The request to beat a "lower bound" is therefore interpreted as a request to beat this empirical benchmark. We improve two coordinates separately; we do not establish a universally simplest axiomatization or a program below all three coordinates.

The mathematical advance. The model in Section 3 proves that progress is independent of the other seven native rules. Together with the seven existing independence results, this completes the irredundance analysis of the original eight-rule kernel [3]. Irredundance concerns literal deletion of a rule from that kernel. It does not preclude the already successful rule fusions, changes of language, or alternate bi-interpretations. Primitive recursion equations without induction have a substantial earlier literature; Friedman studied their equational consequences and the effect of successor coverage [7]. The specific claim here is the proved countermodel for the repository's full native scalar syntax, including its substitution requirements; no literature-wide priority claim is made.

2 A complete proof presentation and its arithmetic meaning

2.1 Syntax and all eight primitive rules

There is one numerical sort. The terms are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in h is bound by I; neither n nor a is within that binding. Every body is finite earlier syntax. Equivalently, one can use a staged first-order signature with an operation $I_h(n, a, \vec{p})$ for each fixed body $h(u, \vec{p})$. This does not introduce program-valued variables or a uniform total evaluator for all primitive recursive descriptions.

Native formulas and their ordinary readings are

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A, \qquad Q_y B \equiv B(0) \to \forall y B(y),$$
$$D_{x,y} B \equiv \exists x \big(B(x, 0) \to \forall y B(x, y)\big).$$

Contradiction is designated by $0 = S0$. Contexts are finite lists of retained native formulas. Capture-avoiding substitution and fresh variables are understood. The precise eight-rule kernel $\mathcal{K}$ is the one of `Lean/ConciseIPRA/Native.lean` [2]:

Rule	Entire schema
Hyp	$A \in \Gamma$ implies $\Gamma \vdash A$.
C0	$\Gamma \vdash I(0, a; u.h) = a$.
CS	$\Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u]$.
Tr	From $\Gamma \vdash s = t$ and $\Gamma \vdash h(s) = r$, infer $\Gamma \vdash r = h(t)$, for any one-term template h.
QE	From $\Gamma \vdash Q_y B(y)$ and $\Gamma \vdash B(0)$, infer $\Gamma \vdash B(t)$.
DE	From $\Gamma \vdash D_{x,y} B$ and $\Gamma, Q_y B(b, y) \vdash A$, infer $\Gamma \vdash A$; b is fresh for the retained data and conclusion.
DI	From $\Gamma, B(t, 0) \vdash B(t, Sz)$, infer $\Gamma \vdash D_{x,y} B$; z is fresh for the context, witness t, and passive parameters.
QI	From $\Gamma, E(z) \vdash E(Sz)$, infer $\Gamma \vdash Q_y E(y)$; E is a literal equation and z is fresh for the retained context and passive parameters.

Let $\mathcal{K}^-$ be these same rules with QI deleted. No primitive ordinary logical, successor-injectivity, reflexivity, or further induction axiom is silently added. Weakening, exchange, contraction, cut, and the needed equational rules are admissible. For example, let $d = I(0, t; u.u)$. C0 gives $d = t$; Tr with the identity template and two copies of this equation gives $t = t$. Tr with a constant template then gives symmetry, and the identity template gives transitivity after one symmetry. These constructions use no progress rule.

2.2 The ordinary logical interface is explicit

For clarity, the ordinary connectives need not be taken on trust as additional rules. Write $C(v, a, b) = I(v, a; u.b)$, so that $C(0, a, b) = a$ and $C(Sv, a, b) = b$. After replacing every auxiliary Q by a D with unused witness, define a formula gate by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(D_{x,y} B) := D_{x,y} G_v B.$$

Renaming prevents capture. The inherited case-rule derivations give $G_0 A$, $G_{Sv} A \dashv\vdash A$, and $A \dashv\vdash D_{r,y} G_y A$ when r, y are fresh.

A selector between equations is $M_v(s = t, r = w) := (C(v, s, r) = C(v, t, w))$. For compound inputs, align their fresh outer binders and define

$$M_v(D_{x,y} A, D_{x,y} B) = D_{x,y} M_v(A, B),$$
$$M_v(E, D_{x,y} B) = D_{x,y} M_v(G_y E, B),$$
$$M_v(D_{x,y} A, E) = D_{x,y} M_v(A, G_y E),$$

where E is an equation. This recursion decreases the maximum binder depth and has $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$. Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. The complete macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= D_{r,z} M_z(A,B), \\ A \wedge_* B &:= D_{r,z} G_z M_{p(z)}(A,B), \\ \exists_* x\, A(x) &:= D_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* D_{r,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned}$$

Here the newly bound variables are fresh and r is unused. The exact derivations, including both formula round trips, are preserved in `reference/foundations/inherited_choice_basis.tex`. This interface uses cases and computations, rather than progress for quantified formulas. QI remains restricted to literal equations; making it available for all matrices would change the intended arithmetic strength.

2.3 Both interpretations and both composites

The ordinary realization T_I is intuitionistic first-order logic with actual equality over the scalar terms. Its nonlogical axioms are C0, CS,

$$\forall x\,(x = 0 \vee \exists y\; x = Sy), \qquad 0 = S0 \to \bot,$$

and contextual induction for literal equations. The genuine falsity bridge is explicit: positive computation equations alone have a singleton model. The native kernel is a proof presentation of this ordinary theory. The readings of Q, D, the logical macros above, and equation induction obtained from QI followed by QE supply translations in both directions [4].

The established arithmetic bridge has the following noncircular order. First construct a scalar predecessor P and prove $P0 = 0$ and $P(Sn) = n$. Next construct a term reflecting actual equality into a Boolean zero test, and compile quantifier-free formulas into characteristic equations. Only then use the resulting quantifier-free induction to prove pairing and projection identities, and simulate general primitive recursion. The unchanged predecessor, reflection, and compilation proofs accompany the archive; they are inherited results, not new claims of this report.

For example, if π, L, H are the proved pairing and projections and

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

the scalar compiler sets

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \quad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the projection identities then prove both defining equations of f^*. Each finite description is compiled externally; there is no single internal evaluator.

Theorem 2.1 (Inherited bi-interpretation, preserved by the refinements)**.** *The ordinary realization checked by the new programs is bi-interpretable and equiconsistent with full first-order* iPRA*. Both numerical domains are whole, equality is actual, and the graph of each numerical composite isomorphism is $x = y$.*

Proof. Interpret each scalar iterator in iPRA by primitive recursion whose update ignores its counter. This preserves its equations, coverage, separation, and equation induction. In the other direction use

the scalar compiler and the predecessor and equality-reflection constructions just specified; they recover the successor axioms and every quantifier-free induction instance, including arbitrary retained contexts.

To check the composites, induct externally on finite operation descriptions. After lower-level operations have been identified, an original operation and its double interpretation satisfy the same initial value and recurrence. Equation induction proves equality at every count. This proves internal uniqueness and identifies their values over the entire domain. Formula composites follow by structural induction from the stated readings and the logical-macro round trips. The identity graph is thus total, bijective, and preserves all interpreted operations. The explicit falsity bridge transfers proofs of contradiction in both directions.

The implementation refinements below preserve exactly the same canonical public theorem set, by exact trace equivalence or canonical proof compilation. Composing those transformations with the inherited interpretations proves the assertion for the new checkers. No theory is claimed to prove its own consistency. □

3 Independence of progress: a coherent model of scalar computation

Theorem 3.1 (Progress is not admissible)**.** *The kernel $\mathcal{K}^-$ does not derive*

$$Q_y\big(I(y,0;u.u)=0\big).$$

Its base instance and successor-step sequent are derivable in $\mathcal{K}^-$. There is a classical countermodel with actual equality, successor injectivity, coverage, separation, and every finite scalar computation. Consequently QI is independent of the other seven native rules.

The following proof takes place in classical metamathematics with choice. Its conclusion is a countermodel to an intuitionistic calculus; using a classical structure as an intuitionistic countermodel is sufficient. No claim of a smaller complete axiomatization follows from this result.

3.1 The semantic issue that must be addressed

An arbitrary choice of an orbit for each *written* update body is not enough. Substituting terms for the passive parameters can change the written body without changing its unary function. We therefore build an *extensional family* of scalar iterators. This is a stronger semantic requirement than merely imposing each displayed computation equation, and it prevents a gap in the native substitution lemma.

A body descriptor is a finite first-order term $h(u,\vec{p})$ with one distinguished state variable u and a finite tuple of passive variables. When an iterator for this descriptor is present, write its function symbol as $I_h(n,a,\vec{p})$. Its computation axioms are

$$I_h(0,a,\vec{p}) = a, \tag{1}$$

$$I_h(Sn,a,\vec{p}) = h(I_h(n,a,\vec{p}),\vec{p}). \tag{2}$$

For *every pair* of available descriptors, including two copies of the same descriptor with different parameter values, require the first-order schema

$$\forall\vec{p}\forall\vec{q}\big[(\forall u\; h(u,\vec{p}) = g(u,\vec{q})) \longrightarrow \forall n\forall a\; I_h(n,a,\vec{p}) = I_g(n,a,\vec{q})\big]. \tag{Ext}$$

Thus an available iterator depends on the actual unary function and the two numerical inputs, rather than on a particular presentation of that function. Parameter repetition, omission, and reordering are included.

3.2 The orbit supplied by one ultrapower

Let A be any nonempty structure, let U be a nonprincipal ultrafilter on $\mathbb{N}$, and put $B = A^{\mathbb{N}}/U$. Fix a unary term function of B,

$$f(v) = h^B(v, \vec{p}), \qquad \vec{p} = [\vec{p_i}],$$

and an element $a = [a_i]$ of B. Write $f_i(v) = h^A(v, \vec{p_i})$. For each ordinary integer $j \in \mathbb{Z}$, set

$$x_j = [f_i^{\max(0,i+j)}(a_i)].$$

For all sufficiently large i, $\max(0, i + j + 1) = \max(0, i + j) + 1$. Hence

$$f(x_j) = x_{j+1} \quad (j \in \mathbb{Z}).$$

Every parameter-definable unary term function in this ultrapower therefore has a full two-sided orbit. The iterations in the coordinates are finite external iterations; no arithmetic induction axiom in A is being used. The displayed sequence need not pass through a, and no such condition is required on a successor component that does not contain zero.

3.3 Extension of a coherent partial family

Suppose A has successor injectivity, zero outside the image of successor, zero/successor coverage, and no finite successor cycles. Suppose it also has a collection of iterator symbols satisfying (1), (2), and all instances of (Ext) for that collection.

Take the ultrapower B in the language containing *all* those symbols. All the stated first-order conditions transfer by Łoś's theorem. In particular, the old iterator values are retained, and their coherence holds also for parameter tuples newly present in the ultrapower.

The successor graph of B consists of the standard ray $0, S0, S^20, \ldots$ and components isomorphic to $\mathbb{Z}$. Indeed, every nonzero point has a unique predecessor. A point outside the standard ray cannot reach zero after finitely many predecessor steps; injectivity and the absence of finite cycles ensure that all steps in its two-sided component are distinct. Choose an origin in each such component.

Now add iterators for any set of descriptors in the *old language*. Partition their parameter instances by the actual unary functions $f : B \to B$ that they define.

If a function f already has an iterator from the old family, use that iterator at every counter and every seed. This prescription is well-defined: if $f = h^B(-, \vec{p}) = g^B(-, \vec{q})$ for two old descriptors, Ext says that their iterator values agree for all counters and seeds.

For a function not represented in the old family, make one simultaneous choice for its entire equivalence class. At a standard counter S^k0 put

$$I_f(S^k0, a) = f^k(a).$$

On a nonstandard successor component with chosen origin b, choose a two-sided orbit $(x_j)_{j\in\mathbb{Z}}$ supplied by the preceding lemma, and set

$$I_f(S^jb, a) = x_j.$$

Here negative powers mean the unique iterated predecessor on that component. Choices may be made separately for each f, seed a, and component. The set-theoretic axiom of choice suffices for all these choices.

Define every newly added $I_h(n, a, \vec{p})$ by the chosen iterator for the actual function $h^B(-, \vec{p})$. Its two computation equations hold by construction. Ext holds for every pair of old or new symbols, because equal unary functions were assigned the same entire iterator. The expansion changes no old function or old element.

3.4 A seed model in which identity iteration is not constant

Start with the disjoint union

$$A_0 = \mathbb{N} \sqcup \mathbb{Z}.$$

Zero belongs to the first summand; successor is ordinary successor on each summand. Let $1 = S0$. Interpret the iterator $J = I_u$ of the identity update by

$$J(n, a) = \begin{cases} a, & n \in \mathbb{N}, \\ 1, & n \in \mathbb{Z}. \end{cases}$$

Then $J(0, a) = a$ and $J(Sn, a) = J(n, a)$. All the required successor conditions hold. The initial extensionality condition for the sole descriptor u is automatic. For any chosen c in the $\mathbb{Z}$ summand,

$$J(c, 0) = 1 \neq 0.$$

No predicate distinguishing the two summands is required in the language.

Iterate the extension construction through ω stages. At each stage, take an ultrapower in the entire current language and add iterators for all finite unary term descriptors of that language. Identify the preceding structure with its diagonal elementary image before adding the symbols. The languages and structures can then be regarded as increasing chains. The reduct at every later stage is an elementary extension in each earlier language. Their union A_ω retains all computation equations, all cross-extensionality schemas, all successor conditions, and the distinguished equation $J(c, 0) = 1 \neq 0$.

To justify the union step explicitly, each fixed old-language sentence is handled by the elementary-chain theorem on the reducts to that language. Every computation or Ext instance uses only finitely many symbols and was imposed at a finite stage, so it holds in the union. Equivalently, Ext can be checked directly: if two final unary functions agree, they agree on every sufficiently late stage containing their parameters; applying Ext at a stage also containing the proposed counter and seed gives equality of the two iterator values there and hence in the union.

3.5 Returning to native scalar syntax

Every finite native term

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h)$$

has an interpretation using finitely many staged symbols. Expand its inner iterators first. The translated update body is then a finite term of some stage language, so its descriptor has an iterator at the next stage. Parameters are ordinary numerical variables; there is no function sort and no internal universal evaluator.

More intrinsically, for any unary function definable by a finite native body with parameters in A_ω, choose a staged descriptor presenting it and use its iterator. Ext makes this choice independent of the descriptor. It follows by structural induction on native terms that capture-avoiding substitution has the usual semantics:

$$[\![t[s/x]]\!]_\rho = [\![t]\!]_{\rho[x \mapsto [\![s]\!]_\rho]}.$$

At an iterator, the induction hypothesis identifies its counter, seed, and pointwise update function; Ext identifies the resulting iterators. This is the exact point at which mere independent choices for written bodies would have been insufficient.

3.6 Soundness without progress and the refuted instance

Interpret native equality as actual equality, and use the ordinary readings

$$Q_y B = B(0) \to \forall y\, B(y), \qquad D_{x,y} B = \exists x (B(x,0) \to \forall y\, B(x,y)).$$

Hypothesis, equality transport, the two computations, closure elimination, and witness elimination are sound. Case introduction is sound because every element is zero or a successor: from $B(t,0)$ and all successor instances $B(t,Sz)$ one obtains every instance. The usual freshness conditions ensure that t and the retained assumptions do not depend on the fresh z. All these arguments are ordinary first-order soundness arguments in the single structure A_ω.

Let

$$E(y) \;:\equiv\; I(y,0;u.u) = 0.$$

The zero computation proves $E(0)$. The successor computation and ordinary derived equality reasoning prove

$$E(z) \;\vdash\; E(Sz)$$

without any induction, since $I(Sz,0;u.u) = I(z,0;u.u) = 0$ under the displayed hypothesis. Nevertheless $E(c)$ is false in A_ω. Consequently $Q_y E(y)$ is false there and is not derivable by the kernel without progress. Thus QI by progress is not admissible over the other seven rules. With an unused existential witness, the same model also refutes the witnessed progress conclusion $D_{x,y} E(y)$.

Proof status and limits. This is a written model construction, not a proof-assistant verification. The essential requirements are simultaneous extensional choice for actual unary functions; universal cross-extensionality for all parameter tuples; ultrapowers in the full current language; and adding only prior-language body descriptors at a given stage. Omitting any of those requirements would leave a genuine coherence or circularity gap. The result proves independence of progress. It does not prove a complexity lower bound for other presentations, and it does not satisfy a requirement for a smaller complete proof checker by itself.

3.7 A stronger initial structure with cancellative addition

The initial structure can already satisfy additive cancellation and the identity law for successor iteration. Take

$$M_0 = \{(0,n) : n \in \mathbb{N}\} \cup \{(k,z) : k \in \mathbb{N}_{>0},\ z \in \mathbb{Z}\},$$

with componentwise addition, $0 = (0,0)$, $1 = (0,1)$, and $S(k,z) = (k, z+1)$. It is a submonoid of the additive group $\mathbb{Z}^2$, so addition is associative, commutative, and cancellative, with identity 0. It is conical: $a + b = 0$ implies $a = b = 0$. Successor is injective, omits zero, has no finite cycle, and every nonzero element has a predecessor in the domain.

Preinstall two iterator symbols:

$$J(n,a) = \begin{cases} a & \text{if the first coordinate of } n \text{ is } 0, \\ 1 & \text{otherwise,} \end{cases} \qquad K(n,a) = a + n.$$

They represent the updates $u \mapsto u$ and $u \mapsto Su$, respectively. Both satisfy their zero and successor computations. The added identity law $K(n,0) = n$ holds, as do the usual additive zero and successor laws. Their cross-extensionality condition has a false antecedent, since identity and successor are distinct functions. Extensionality within each description is immediate. Thus this is a valid starting point for the same extension tower. All these first-order laws survive that tower.

At $c = (1, 0)$, however, $J(c, 0) = 1$. Hence the final countermodel can have all of the listed additive laws, including cancellation, and the successor identity orbit, while still refuting literal zero-equation induction.

This does not furnish a countermodel to full successor uniqueness. Indeed, already in the initial structure the term

$$f(n) = n + J(n, 0)$$

satisfies $f(0) = 0$ and $f(Sn) = Sf(n)$, whereas $f(c) = Sc \neq c = K(c, 0)$. Its addition is itself the successor-iterator operation K, so this failure uses scalar terms. Full successor uniqueness therefore fails in this structure, as it must.

4 Finite scalar clocks without induction

This section resolves the even-cycle gap in the linked README's first further question as a written theorem. It does not remove the progress rule or establish a smaller complete checker. The result is a constraint on any proposed countermodel for the independence of progress.

Work first in intuitionistic first-order logic with actual equality, the two scalar iterator computations, and numerical coverage

$$x = 0 \ \vee \ \exists y \, x = Sy.$$

There is no induction rule. Write

$$C(x, a, b) = I(x, a; u.b), \qquad Z(x) = C(x, 1, 0), \qquad \bar{j} = S^j 0.$$

Thus $C(0, a, b) = a$, $C(Sx, a, b) = b$, and coverage proves $Z(x) = 0 \vee Z(x) = 1$. All inductions below are external inductions on a fixed finite integer specifying a term, not induction in the object theory. For a fixed unary term h, $h^{[r]}$ means r-fold syntactic composition; it does not introduce a new iterator or an object-language quantifier.

Lemma 4.1 (Finite clocks)**.** *For every fixed integer $m \geq 2$, there is a finite scalar term $h_m(x)$ such that the theory without induction proves*

$$h_m(x) \in \{\bar{0}, \ldots, \overline{m-1}\}, \qquad h_m(\bar{j}) = \overline{j+1} \quad (0 \leq j < m-1), \qquad h_m(\overline{m-1}) = 0.$$

Here membership abbreviates a finite disjunction of actual equalities.

Proof. Start with $h_2 = Z$. Given h_m, define

$$T_m(x) = I(x, 0; u.h_m(u)).$$

Coverage on x and the two computations show that $T_m(x)$ belongs to $\{\bar{0}, \ldots, \overline{m-1}\}$: the zero case is 0, and in the successor case its value is an h_m-value. No assertion about the preceding iterator value is needed.

For $1 \leq r < m$, put

$$d_{m,r}(z) = Z\big(h_m^{[m-r]}(z)\big), \qquad V_{m,0}(z) = 0,$$
$$V_{m,r}(z) = C\big(d_{m,r}(z), V_{m,r-1}(z), \overline{r+1}\big), \qquad V_m = V_{m,m-1}.$$

Finite ground computation proves $d_{m,r}(\bar{s}) = 1$ for $s = r$, and $d_{m,r}(\bar{s}) = 0$ for $0 \leq s < m$, $s \neq r$. Consequently

$$V_m(0) = 0, \qquad V_m(\bar{r}) = \overline{r+1} \quad (1 \leq r < m).$$

Every value of V_m, at an arbitrary argument, is one of $0, \bar{2}, \ldots, \bar{m}$, by finitely many applications of coverage to the selector counters. Define

$$\boxed{h_{m+1}(x) = C\big(x, 1, V_m(T_m(x))\big).}$$

Its range is $\{0, \bar{1}, \ldots, \bar{m}\}$. At $x = 0$ its value is 1. At $x = \bar{j}$ with $1 \le j < m$, finite computation gives $T_m(\bar{j}) = \bar{j}$, so its value is $\overline{j+1}$. At $x = \bar{m}$, $T_m(\bar{m}) = 0$, so its value is 0. This completes the external construction. □

The first nontrivial extension has the compact description

$$h_3(x) = C\big(x, 1, C(Z(Z(T_2(x))), 0, 2)\big), \qquad T_2(x) = I(x, 0; u.Z(u)).$$

Direct computation gives $h_3(0) = 1$, $h_3(1) = 2$, and $h_3(2) = 0$.

Lemma 4.2 (Separation of finitely many numerals)**.** *For fixed $m \ge 2$ and distinct $i, j < m$, the assumption $\bar{i} = \bar{j}$ proves $0 = 1$, without induction or successor injectivity.*

Proof. Apply $h_m^{[r]}$ with $r \equiv -i \pmod m$ and $0 \le r < m$. The resulting equation is $0 = \bar{s}$ for some $1 \le s < m$. Applying Z gives $1 = 0$, whose symmetric form is the desired contradiction. Only finitely many computation equations are used. □

Theorem 4.3 (No finite successor cycle without induction)**.** *For every fixed integer $k \ge 1$ and every scalar term c, the theory without induction derives*

$$S^k c = c \;\longrightarrow\; 0 = 1.$$

With $0 \ne 1$ it therefore proves $S^k c \ne c$.

Proof. Choose $m = k+1$ and set $v = T_m(c)$. The hypothesis and k applications of successor computation give

$$v = T_m(c) = T_m(S^k c) = h_m^{[k]}(v).$$

The finite-range property of T_m gives the cases $v = \bar{j}$ for $0 \le j < m$. In each case the displayed equation becomes

$$\bar{j} = \overline{j + k \pmod m}.$$

Since $k = m - 1$, the two residues differ. The preceding separation lemma gives $0 = 1$ in every case. □

4.1 Removing the finite disjunctions in the native calculus

There is also a direct written translation into the native kernel without progress, so the preceding use of ordinary disjunction is dispensable. The existing theorem `NoInd.cases` derives the following rule for equation templates: if $E(0)$ and $E(Sz)$ are derivable uniformly in the retained context, then $E(t)$ is derivable for every term t. The computation and crossed equality rules provide ordinary equational congruence and transitivity without progress.

Call a term a finite selector tree over L if it is built from leaves in the fixed finite list L by $C(q, a, b)$, with arbitrary counter terms q. For any equation template E, if $E(\ell)$ is derivable uniformly for each leaf $\ell \in L$, then $E(t)$ is derivable for every such tree t. Prove this externally by induction on its finite syntax: at a selector, apply `NoInd.cases` to its counter and compute the two branches. Each $h_m(x)$ is literally a finite selector tree over $L_m = \{\bar{0}, \ldots, \overline{m-1}\}$. A further application of `NoInd.cases` to the counter of $T_m(c)$ shows:

$$\big(E(\bar{j}) \text{ derivable for every } j < m\big) \quad\Longrightarrow\quad E(T_m(c)) \text{ derivable.} \tag{FR}$$

The zero branch is $E(0)$; the successor branch is $E(h_m(T_m(z)))$, handled by the selector-tree argument.

Put $b_r(a) = Z(h_m^{[(-r \bmod m)]}(a))$ for $0 \le r < m$. These are Boolean terms, with $b_r(\bar{j}) = 1$ exactly when $r = j$. For Boolean arguments define

$$\mathrm{xor}(p, q) = C(p, q, Z(q)), \qquad \mathrm{or}(p, q) = C(p, q, 1).$$

Let $D_m(a, b)$ be the left-associated finite disjunction (starting at 0) of the terms $\mathrm{xor}(b_r(a), b_r(b))$. Finite zero/successor cases prove $D_m(a, a) = 0$ for every a; no disjunction formula or equality characteristic for arbitrary numbers is used. For $k = m - 1$, ground computation proves

$$D_m(\bar{j}, h_m^{[k]}(\bar{j})) = 1 \qquad (j < m).$$

Apply (FR) to this equation template to obtain $D_m(v, h_m^{[k]}(v)) = 1$ at $v = T_m(c)$. The cycle assumption gives $v = h_m^{[k]}(v)$, so equational transport and $D_m(v, v) = 0$ give $0 = 1$. This is the required native derivation, with no progress rule.

Boundary of the result. The construction and the native translation are written proofs; no new Lean verification is claimed. They do not prove the progress rule redundant, and do not by themselves settle whether successor uniqueness is equivalent to full iPRA. The novelty is the general finite-clock construction, which completes the existing odd-cycle exclusion.

The companion `check_finite_clocks.py` builds genuine scalar terms using de Bruijn variables and evaluates them on bounded standard inputs. It stores repeated subexpressions as a directed acyclic graph; this is only a representation of a finite expanded term. It represents $h_m^{[r]}(a)$ as $I(\bar{r}, a; u.h_m(u))$, which expands to the same composition by finitely many computations. Its default run checks periods 2 through 12 and numerical inputs 0 through 32, including finite range, clock values, cycle transitions, and the Boolean discriminator: 1,243 checks pass. These evaluations corroborate finite examples only; the external construction and proofs above establish the universal theorem.

5 Smaller implementations of the complete calculus

5.1 The measured public contract

The compact programs take a finite JSON certificate consisting of integers and arrays, with a canonical closed guarded Eq/D public goal. Intermediate proofs use the inherited larger auxiliary language. A guard replaces an equation $s = t$ below a binder by

$$I(y, 0; u.s) = I(y, 0; u.t),$$

under a closure with unused witness. Its zero instance is reflexive; its successor instances express $s = t$. This is why restricting public atomic matrices to guarded ones preserves the ordinary theory while permitting a smaller root-selected introduction [5].

The compact modules do not themselves reject every malformed public grammar or every non-integer JSON leaf. The 869-byte strict companion includes those checks, including unused certificate prefixes. It is a separate program under a stronger input contract. A strict program's size must not be compared as though it had the same job as the compact 506-byte control. All programs import only the Python JSON library; no arithmetic oracle, local checker module, embedded compressed program, `eval`, or `exec` is used.

The inherited arity codec stores a term or formula as an integer/list tree, using weighted binding slots. Its common opener is denoted below by $J_r(t)$: opening tree t with image r at formula depth zero and the current numerical scope. Its recursive depth increment is $5 - \ell$, where ℓ is the node

arity. Canonical numerical constructors have odd arity 1 or 3. Equations have arity 2, paired formulas arity 1, and auxiliary closures arity 3. Full decoding and compilation specifications are included in `reference/foundations/inherited_arity_checker.tex` and `reference/COMPACT_CONTRACT.txt`.

5.2 The 251-token refinement is trace-equivalent

The 253-token control reads a four-field instruction as

```
m, s, u, t = q()
```

The new program reads

```
m, *r = q()
s, u, t = r
```

and reuses r where the control constructs `[s,u,t]`, and `r[:2]` where it constructs `[s,u]`. The second unpack enforces exactly three tail fields, so the required instruction arity remains four. If both unpacks succeed, the variables have exactly their former values and r is exactly the former constructed list. No scope, context, dispatch test, stream consumption, or inference changes. If either unpack fails, both programs reject. Thus successful traces agree for every finite JSON certificate, apart from finite runtime resource limits. The measured saving is two significant tokens, with a cost of three AST nodes and six normalized bytes relative to its matched control. This is an implementation refinement, not a new inference rule.

5.3 The 502-byte refinement and its completeness argument

The 506-byte control already fuses continuation opening, allocation of a new parameter, and context extension. Its continuation selector has the form

$$\mathcal{C} \ \texttt{and} \ J_s(t) \ \texttt{or} \ a, \quad \mathcal{C} := ((m,a) = (M, J_u(t))),$$

where M is the required equality or closure major. The 503-byte parity companion selects instead from the tuple $(a, J_s(t))$ with Boolean index $\mathcal{C}$. This eagerly evaluates $J_s(t)$.

For integer/list data, whenever the eager evaluations succeed, the two values coincide. If $\mathcal{C}$ is false both select a. If it is true and $J_s(t)$ is truthy, both select $J_s(t)$. If $J_s(t)$ is false, then the tree has an empty-list or scalar root: a successful opening of a nonempty list is nonempty. At formula depth zero those empty/scalar openings do not depend on their proposed image, so $J_s(t) = J_u(t) = a$, and the selectors still agree. The eager companion can reject additional noncanonical certificates, but cannot create a new successful trace relative to the parity control.

Canonical completeness survives eager evaluation. Genuine elimination already requires the source opening. In a computation packet the otherwise inspected field is a numerical update body; in an introduction packet it is a numerical witness. Starting at depth zero, their odd node arities keep every encountered depth even. At an integer leaf the opener either retains a legal coordinate or returns padding zero, without opening the proposed image. Thus the extra evaluation terminates on every such canonical field. Unused fields of witness-elimination packets can be set to zero. This is a standardization of certificate data, not a restriction of the theorems that have certificates.

The final byte replaces `~len(a) % 2` by `3 % len(a)`, as described in the incoming smaller-envelope report. On the meaningful matrix arities the values are identical:

Matrix	D (arity 1)	Equation (arity 2)	Q (arity 3)
Both mode values	0	1	0

An empty matrix now rejects by division by zero. Surplus arities in the permissive administrative decoder mean the true equation $0 = 0$; a changed mode there affects only a truth alias or the substitution instance of a computation equation. No false public consequence is introduced. This last argument uses the inherited sorted decoder, so the 502-byte program is not claimed to have exactly the same raw acceptance language as the control. The 503-byte parity companion is supplied for the cleaner trace inclusion statement.

Freshness is retained. The expression `k := ~-k` is inherited from the 506-byte control and means $k - 1$. Source and target openings in continuation selection occur before allocation; the outer opening and context extension see the new scope. The new programs preserve this order. Replacing the scope counter naively by context length would get that phase boundary wrong; none of the delivered checkers makes that replacement.

5.4 The strict companion

After JSON parsing, the strict module first checks that the printed value uses only spaces, brackets, commas, minus signs, and decimal digits. On parsed JSON values, this character test accepts precisely integer/list trees: strings introduce quotes; objects braces or colons; Boolean values and null introduce letters; non-integral numbers introduce a decimal point or letters. The public grammar validator then checks Eq/D arities, numerical constructors, the permitted numerical slot form, and literal guards immediately below a paired binder. The kernel's initial opening checks the numerical scope. These checks are inside the measured module, not external preconditions of the strict companion.

5.5 Executable evidence and its boundary

The delivered `checker/reproduce.py` measures every whole module, checks frozen source identities, and replays 512 checker processes under ordinary and optimized Python. Every observed outcome matches its stated expectation. Fixtures include genuine uses of an induction hypothesis, arbitrary specialization, future-variable capture, context leaks, false goals, surplus arities, erased redundant fields, and malformed prefixes or packets. Global-scope token fixtures and local-scope byte fixtures are separate because they use different certificate conventions.

The benchmark audit measures all 1,416 Python files in the three incoming archives, including the 1,380 distinct frozen source blobs of the prior Research audit. Programs below the previous byte record were inspected as utilities, not standalone theorem checkers. The current Research tree identity and all classification receipts are in `checker/evidence/`. The record claim is limited to this inspected corpus and its documented eligibility rules.

Finite tests verify concrete executions. The universal mathematical arguments are the trace and compiler proofs above and the inherited arithmetic bridge. The nonstandard countermodel is not simulated or established by these tests. No new Lean or Rocq verification is asserted.

6 What the new mathematics permits us to conclude

The independence theorem closes a specific gap in the current research program. The equation $I(y, 0; u.u) = 0$ is an explicit witness that all scalar computations, equality, cases, and the ordinary Q/D rules do not force equation induction. The countermodel is acyclic, has actual equality, and even admits a cancellative addition with the usual elementary identities. It does not exploit an ill-typed formula, a failed successor cover, a finite cycle, or incompatible interpretations of substituted bodies.

Combined with the seven independent deletions already established in `NativeModels.lean`, this yields an eight-out-of-eight irredundance result for the original rule inventory. The eight rules are not a

global minimum: the repository itself has equivalent presentations with fewer rule families, obtained by fusion or a changed syntax. Likewise, the smaller token and byte programs do not establish an absolute minimum description length.

For the requested axiomatization, the honest outcome is therefore a complete inherited bi-interpretable presentation with two smaller implementations, together with a mathematical theorem showing why one tempting deletion cannot work literally. A new mathematical axiom basis with a single complete program below all three current coordinates remains unconstructed.

7 Further research questions

1. Can one complete program beat 251 significant tokens, 275 AST nodes, and 502 normalized bytes simultaneously under a fixed common input contract? A joint result must be one measured program, not an envelope assembled from several files.
2. Can the ultrapower extension lemma be formalized in the existing Lean development using arbitrary carriers and interpreted successor? The current natural-number readings fix the external standard successor and therefore cannot witness the new independence result.
3. Can a different presentation combine computational equality with a weaker induction interface without reintroducing its complexity in syntax validation? The independence proof rules out deletion in the fixed kernel; it leaves such a redesign open.
4. Which fragments of the coherent extension lemma survive if additional iteration laws, such as fusion or uniqueness, are imposed? Computation and extensionality of the update alone are compatible with the counterexample. Uniqueness for the identity update immediately excludes its specific witness, but excluding one model does not prove full arithmetic strength.
5. Can a counterpart of the new countermodel decide the remaining successor-uniqueness versus full-equation-induction question? The strengthened model below refutes full successor uniqueness and so is not a negative answer to that different question.

A Transfer and the countable-model refinement

For completeness, the model construction uses ordinary ultrapowers, not set-theoretic large-cardinal ultrapowers. Fix a nonprincipal ultrafilter U on the ordinary natural numbers, available in the classical metatheory with choice. Elements of $A^{\mathbb{N}}/U$ are sequences modulo agreement on a set in U; operations are interpreted coordinatewise. For every first-order formula φ,

$$A^{\mathbb{N}}/U \models \varphi([\vec{a}_i]) \quad \Longleftrightarrow \quad \{i : A \models \varphi(\vec{a}_i)\} \in U.$$

Here is the needed proof outline. The atomic case is the definition of the quotient and of coordinatewise operations. Finite Boolean connectives use the ultrafilter laws. For an existential formula, a witness sequence gives the forward direction; in the other direction choose a witness in each coordinate where one exists, and an arbitrary element elsewhere. This proves the statement by structural induction on formulas. In particular the diagonal embedding by constant sequences is elementary. This is the usual proof of Łoś's theorem.

The language at each stage is countable, and so is the union language. Although the constructed union may be uncountable, it has a countable elementary submodel containing the tracked element c. One may construct such a submodel by starting with c and the named constants, closing under the countably many function symbols, then choosing witnesses for every existential formula with parameters already selected, and repeating this countably many times. The witness property gives an elementary submodel by induction on formulas. It retains every computation and Ext sentence as well as $J(c, 0) = 1 \neq 0$.

Thus the countermodel can be made countable. This does not contradict the repository's obstruction to standard-natural-number readings: the successor on this countable carrier has nonstandard components and is not the externally fixed map $n \mapsto n + 1$ after an arbitrary enumeration. The proof does not claim a computable presentation of the full model.

B Complete compact checker listings

The following are complete programs, including input parsing and the success signal. Their larger strict companion and frozen controls are separate files in the archive. Acceptance means normal termination with exact output `ACCEPT` and a newline; exceptions reject. As with the comparison programs, the theoretical algorithms allow unbounded finite resources, while a finite Python recursion or memory limit can reject a sufficiently large certificate.

B.1 251-token program

```
from json import load
k = 0

def i(a, t=0j, d=0):
    try:
        return [i(b, t, d + 5 - len(a)) for b in a]
    except:
        return a if k <= a < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    while i(a) not in g:
        m, *r = q()
        s, u, t = r
        if i(t, u) == a:
            m = r[:2] if s else [t] * 3
            a = i(t, s)
        h = c(m, g) or m * 3 if m else [[s and [s], u, t], i(t, r, 3) if s else u] if u else (a := i(*a, t, 2))
        k -= 1
        g += (i(h, (m or len(a) == 2) and k),)
        a = i(a, [k])
q = load(open(0)).pop
c(q(), ())
print('ACCEPT')
```

B.2 502-byte program

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d else d % 2 and
      i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t, 2))
    c(i((a, i(t, s))[(m, a) == (s and [s, u] or [t] * 3, i(t, u))], [(k := (~-k))]), g + [i(h, (m or 3 % len(a))
     and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

References

[1] V. Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, commit `6ffc49f55d355b548f7a8e1530570aee5de086b3`, 12 November 2026. Pinned project README, especially Further questions 1.

[2] ProveIt, `Lean/ConciseIPRA/Native.lean` and `NativeEq.lean`, same snapshot. Explicit eight-rule kernel.

[3] ProveIt, `Lean/ConciseIPRA/NativeModels.lean`, same snapshot. Seven rule-independence models and the open progress case.

[4] ProveIt, `Research/certified_zero/reference/inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex`, same snapshot. Whole-domain arithmetic bridge. Unchanged copies accompany this report.

[5] ProveIt, `Research/deterministic_introduction/implementation/code/compact_optional.py`, Git blob `a0af7afb236b530eb30c63c5ab788151024271b7`, and the inherited guarded public-language proofs. Frozen 506-byte comparator.

[6] ProveIt, `docs/incoming/ipra_smaller_envelope.zip`, `ipra_frontier_review_2026-11-12.zip`, and `ipra_orbit_closure.zip`, same snapshot. The first supplies the 253-token and 275-node coordinate records; the third audits the 506-byte source. Pinned intake directory.

[7] H. Friedman, *Provable equality in primitive recursive arithmetic with and without induction*, Pacific Journal of Mathematics **57**(2), 379–392 (1975). Original paper.